

Vansh Mahendru

Sonipat, Haryana · +91-8810599936 · vanshmahendru85@gmail.com · github.com/mygit72 ·
LinkedIn

Education

SRM University

B.Tech in Computer Science and Engineering

Sonipat, India

2023 – 2027 (Expected)

- Cumulative GPA: 7.8/10.0

Relevant Coursework

Data Structures & Algorithms
Operating Systems
Computer Networks
Artificial Intelligence

Object-Oriented Programming
Database Management Systems (DBMS)
Linear Algebra & Probability & Statistics
Software Engineering Principles

Skills

Languages & Tools Python (clean, modular, testable), SQL, C++, Git, GitHub

LLMs & Generative AI Large Language Models (LLMs), RAG Pipelines, LangChain, ChromaDB, Vector Databases, Prompt Engineering, LLM Evaluation & Evals Harnesses, Retrieval Systems, Agent Patterns, Hybrid Search, Embedding Models, OpenAI API, Gemini API

ML Engineering End-to-End ML Pipelines (Preprocessing → Training → Evaluation → Inference), Model Debugging & Optimisation, Production Deployment, Regression Testing

Backend & APIs FastAPI, REST API Design, AI Model Integration, Backend Development, LLMops, Structured Inference Layers

Frameworks & Libs TensorFlow, Keras, Scikit-Learn, XGBoost, Pandas, NumPy, Matplotlib

Cloud & Infra AWS/Azure (Exposure), Model Deployment, Software Development Best Practices

Research Projects

SPLM India: Satellite Parking Lot Momentum | *Python, YOLOv8, NumPy, Pandas, FastAPI* GitHub

- **Objective:** Built a complete, end-to-end ML intelligence pipeline — owned design, data contracts, training, evaluation, and inference — to predict revenue surprises in Indian retail stocks using Sentinel-2 satellite imagery datasets (10,000+ records).
- **Approach:** Wrote clean, modular, testable Python across all pipeline stages; applied RAG-style NLP workflows (chunking, parsing, embedding-style feature extraction) on unstructured feeds; engineered NDBI spectral domain features; exposed inference via a structured, API-compatible scoring interface with defined data contracts.
- **Result:** Achieved 0.72 correlation with revenue targets; full pipeline (preprocessing → training → evaluation → inference) deployed at sub-second latency, yielding 14% annualised alpha — shipped a meaningful, production-ready system end-to-end.

The Lottery Ticket Hypothesis | *Python, Keras, TensorFlow*

GitHub

- **Objective:** Developed an ML model engineering and optimisation framework to iteratively prune neural networks, producing lightweight, production-ready models — a direct analogue of the eval-and-harden loop expected in production LLM systems.
- **Approach:** Implemented iterative pruning pipelines in Keras/TensorFlow with clean, modular code;

wrote systematic regression tests at each sparsity threshold; debugged model failures across training and inference; validated final model against reliability and performance benchmarks for deployment readiness.

- **Result:** Achieved 10× model size reduction at 90% sparsity while maintaining 99.2% MNIST accuracy — demonstrated strong capacity to debug, test, optimise, and ship ML pipelines without sacrificing quality.

Technical Projects

Document Intelligence RAG Chatbot | *Python, LangChain, ChromaDB, OpenAI/Gemini, FastAPI* [GitHub](#)

- **Objective:** Built a production-grade document Q&A system using a full RAG architecture — covering PDF ingestion, chunking, embedding, vector storage, retrieval, and LLM-based generation — with a measurable eval layer to score answer quality, checking every box of a real-world Generative AI deployment pipeline.
- **Approach:** Ingested multi-format PDFs using LangChain document loaders; applied recursive text chunking strategies and OpenAI/Gemini embedding models to convert chunks into dense vector representations; stored and indexed embeddings in ChromaDB with metadata filtering for hybrid search; implemented a retrieval chain (top-*k* similarity + MMR reranking) feeding a generation step with prompt-engineered context windows; built an eval harness using LLM-as-judge patterns to score factuality, relevance, and groundedness of generated answers; exposed the full pipeline as a FastAPI backend with structured request/response contracts.
- **Result:** Delivered an end-to-end RAG system with measurable answer quality metrics; eval layer consistently distinguished high-quality retrievals from hallucinated responses — demonstrating production-level RAG pipeline design, LLM evaluation methodology, and FastAPI deployment in a single project.

Multi-Tool AI Agent with LLMops Deployment | *Python, LangChain, OpenAI Assistants API, FastAPI* [GitHub](#)

- **Objective:** Designed and deployed an autonomous AI agent with multi-tool use capability — web search, calculator, and file reader — using LangChain and the OpenAI Assistants API, packaged with a FastAPI backend to demonstrate agent design, LLMops, and production deployment in one end-to-end project.
- **Approach:** Implemented a ReAct-style agent loop using LangChain's agent framework with three pluggable tools: a web search tool (SerpAPI/Tavily), a Python REPL calculator, and a file reader for structured document ingestion; integrated the OpenAI Assistants API for persistent thread and state management across multi-turn conversations; applied LLMops practices including prompt versioning, tool call logging, failure tracing, and structured output validation; deployed the agent as a FastAPI service with a clean REST interface, streaming support, and structured JSON inference contracts; wrote regression tests to guard against tool-call regressions across agent iterations.
- **Result:** Delivered a fully operational multi-tool agent deployed behind a production FastAPI backend; agent correctly decomposed complex multi-step queries into tool calls, executed them reliably, and synthesised structured final answers — showcasing autonomous agent design, LLMops discipline, and full-stack AI deployment capability in a single project.

F1 Race Prediction & Season Analytics Engine | *Python, XGBoost, Scikit-Learn, FastAPI* [GitHub](#)

- **Objective:** Designed and owned a full ML pipeline and backend analytics engine covering data ingestion, preprocessing, model training, evaluation, and real-time inference for race outcome prediction.
- **Approach:** Built modular, testable Python components across data preprocessing, feature engineering, and model training (XGBoost + Monte Carlo, 10,000+ iterations); used Scikit-Learn cross-validation; integrated a FastAPI inference layer exposing model predictions as structured outputs consumed by downstream visualisation; wrote tests to catch regressions between pipeline stages.
- **Result:** Achieved 82% podium prediction accuracy; clean, separated codebase deployed end-to-end with real-time inference — shipped a production-grade system with measurable quality metrics.

Oceanic Edge: AIS & NLP-Driven Port Congestion System | *Python, NLP, Computer Vision* [GitHub](#)

- **Objective:** Built an AI pipeline integrating NLP-style text processing and computer vision to extract structured intelligence from unstructured AIS data streams and satellite imagery — mirroring the retrieval + agent patterns used in production LLM systems.
- **Approach:** Applied data cleaning, parsing, chunking, and embedding-style feature extraction on raw AIS text feeds (RAG-based NLP workflow patterns); integrated AI model outputs into a backend layer with structured, API-ready inference deployed across 50+ global ports; debugged failures across data, model, and inference layers.
- **Result:** Demonstrated 1.8 Sharpe Ratio in supply chain disruption strategies; end-to-end AI pipeline deployed with continuous real-time inference, validating strong AI model integration, retrieval pipeline design, and production deployment capability.

Brain Tumour Detector — Deep Learning Classification Pipeline | *Python, Keras, FastAPI* [GitHub](#)

- **Objective:** Developed, evaluated, and deployed a deep learning classification pipeline for automated brain tumour detection — designed for direct integration into clinical AI applications, requiring production-grade reliability.
- **Approach:** Owned the full preprocessing → training → evaluation → inference pipeline in Keras/TensorFlow; applied data augmentation and class balancing; exposed the trained model via a FastAPI inference endpoint; built an eval rubric using accuracy, F1-score, precision, and recall to make quality measurable.
- **Result:** Attained 98.4% validation accuracy and 0.96 F1-score; clean, deployment-ready codebase with backend API layer — showcasing end-to-end AI/ML pipeline development with rigorous evaluation and production integration.

Certifications

- | | |
|--|-----------|
| • LangChain for LLM Application Development (<i>DeepLearning.AI / Andrew Ng</i>) | Mar 2026 |
| • Deep Learning — Neural Networks with TensorFlow (<i>Packt/Coursera</i>) | Dec 2025 |
| • Machine Learning with Python (<i>IBM/Coursera</i>) | Nov 2025 |
| • Introduction to Artificial Intelligence (AI) (<i>IBM/Coursera</i>) | Nov 2025 |
| • Python for Data Science and AI (<i>IBM/Coursera</i>) | Sept 2025 |